

PaySim Fraud Analysis

Silver Layer — Validation & Fraud Signal Engineering

By Mary Kane

The Silver layer focuses on transforming raw transaction data into validated fraud indicators.

This stage introduces transaction-level logic to identify suspicious behavior patterns and prepare the dataset for business-ready fraud scoring.

```
In [1]: # Importing Libraries
import pandas as pd
import numpy as np
```

```
In [2]: import os
```

```
In [3]: os.getcwd()
```

```
Out[3]: 'C:\\Users\\ZachG'
```

```
In [4]: silver = pd.read_csv("OneDrive/Desktop/Data Analysis/PAYSIM PROJECT/01_data/bronze_
```

```
In [5]: silver.head()
```

```
Out[5]:
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703



```
In [6]: # Creating working Copy
silver = silver.copy()
```

Silver Layer Objective

The purpose of the Silver layer is to create validation logic and engineered fraud indicators.

This stage converts raw transactional fields into investigation-ready features that support fraud detection workflows.

Clusters: Creating Amount Buckets

```
In [7]: silver["amount_bucket"] = pd.cut(
        silver["amount"],
        bins=[0, 100, 1000, 10000, 100000, 1000000, silver["amount"].max()],
        labels=["0-100", "100-1K", "1K-10K", "10K-100K", "100K-1M", "1M+"]
    )
```

```
In [8]: # Testing
        silver["amount_bucket"].value_counts()
```

```
Out[8]: amount_bucket
100K-1M      2706696
10K-100K     2239207
1K-10K       1143445
1M+          130626
100-1K       128541
0-100        14089
Name: count, dtype: int64
```

Transaction amounts were grouped into buckets to evaluate whether fraud clusters around specific value ranges.

Bucketing reduces skewness and improves pattern interpretation.

Origin Balance Validation

```
In [9]: # Old balance origin - transaction amount ~ New balance
        silver["origin_balance_change"] = (
            silver["oldbalanceOrig"] - silver["newbalanceOrig"]
        )
```

```
In [10]: # Creating mismatch Flag
        silver["origin_balance_mismatch"] = (
            round(silver["origin_balance_change"], 2) != round(silver["amount"], 2)
        )
```

```
In [11]: # Test
        silver["origin_balance_mismatch"].value_counts()
```

```
Out[11]: origin_balance_mismatch
True      5125552
False     1237068
Name: count, dtype: int64
```

Destination Balance Validation

```
In [12]: # New balance Dest - Old balance Dest ~ amount
        silver["destination_balance_change"] = (
            silver["newbalanceDest"] - silver["oldbalanceDest"]
        )
```

```
In [13]: # Mismatch flag
silver["destination_balance_mismatch"] = (
    round(silver["destination_balance_change"], 2) != round(silver["amount"], 2)
)
```

```
In [14]: # Test
silver["destination_balance_mismatch"].value_counts()
```

```
Out[14]: destination_balance_mismatch
True      6362604
False         16
Name: count, dtype: int64
```

Balance validation checks were created to compare expected account movement against recorded transaction values.

Mismatch indicators may reveal irregular transaction behavior or incomplete balance movement.

High Risk Transaction Flag

Previous Bronze findings showed fraud exists only within TRANSFER and CASH_OUT categories.

A high-risk transaction flag was created to identify these transaction types.

```
In [15]: high_risk_types = ["TRANSFER", "CASH_OUT"]

silver["high_risk_type_flag"] = (
    silver["type"].isin(high_risk_types)
)
```

```
In [16]: # Test
silver["high_risk_type_flag"].value_counts()
```

```
Out[16]: high_risk_type_flag
False    3592211
True      2770409
Name: count, dtype: int64
```

Large Transaction Flag

```
In [17]: # Top 5% amount threshold:
large_threshold = silver["amount"].quantile(0.95)
```

```
In [18]: # Creating Flag
silver["large_transaction_flag"] = (
    silver["amount"] >= large_threshold
)
```

```
In [19]: # Test
silver["large_transaction_flag"].value_counts()
```

```
Out[19]: large_transaction_flag
False    6044489
True      318131
Name: count, dtype: int64
```

Approximately 318,131 transactions were classified as large transactions based on the top transaction percentile threshold.

Most transactions fall below the large transaction threshold, indicating that unusually large amounts represent a relatively small portion of total activity.

Large transactions alone are not sufficient to identify fraud, but they may contribute to elevated risk when combined with other indicators.

FRAUD VALIDATION TABLE

```
In [20]: silver.groupby("isFraud")[
          "origin_balance_mismatch",
          "destination_balance_mismatch",
          "high_risk_type_flag",
          "large_transaction_flag"
        ].mean()
```

	origin_balance_mismatch	destination_balance_mismatch	high_risk_type_flag	large_t
isFraud				
0	0.806607	1.000000	0.43469	
1	0.005479	0.998052	1.00000	



The validation table compares engineered fraud indicators across fraudulent and non-fraudulent transactions.

Analyst Interpretation

Silver Layer Findings

Fraudulent transactions show strong concentration within high-risk transaction categories.

Large transaction amounts occur more frequently among fraud cases, but transaction size alone does not explain fraud behavior.

Validation indicators suggest fraud emerges through combinations of transaction context rather than a single isolated factor.

This supports moving toward a composite fraud scoring model in the Gold layer.

Silver Layer Summary

Key Silver findings include:

- Fraud occurs entirely within high-risk transaction types
- Large transaction amounts increase fraud likelihood
- Balance validation provides additional fraud context
- Fraud appears to depend on multiple interacting indicators
- Engineered features are now ready for fraud scoring

The next step is to create the Gold layer, where fraud signals will be combined into risk categories and dashboard-ready metrics.

```
In [22]: # Saving Silver Layer
silver.to_csv(
    r"C:/Users/ZachG/OneDrive/Desktop/Data Analysis/PAYSIM PROJECT/01_data/silver_c
    index=False
)
```

```
In [ ]:
```